

End user's guide

Table of contents:

1. How to kick-start	- p 2
2. Overview of scheduler commands (SLURM)	- p 2
3. Loading and Unloading Modules	- p 3
4. srun	
4.1 Simple job with srun	- p 4
2.2 srun job with GPU	- p 4
4.3 srun job with several cores and GPUs	- p 5
5. sbatch	
5.1 Simple job with sbatch	- p 5
5.2 sbatch job with GPU	- p 6
5.3 sbatch job with several cores and GPUs	- p 6
6. Using Jupyter	
6.1 Jupyter Notebook examples	- p 7
7. MPI examples:	
7.1 OpenMPI sbatch job	- p 8
7.2 MVapich2 srun job	- p 10
7.3 MVapich2 sbatch job	- p 11

1. How to kick start

On a Windows operating system, an SSH client such as PuTTY (<http://www.putty.org>) can be downloaded. A Mac OS X user can use the Terminal application from the Finder, or under Application/Utilities/Terminal.app.

When using the SSH connection, the cluster's address must be added. When the connection is made, a username and password must be entered at the prompt.

Optionally, after logging in, the password used can be changed using the passwd command:

```
$ passwd
```

2. Overview of scheduler commands (SLURM)

srun – submit interactive task

sbatch – submit batch script to SLURM

srun and **sbatch** important flags to remember:

- p - partition to use. Please use “-p *partition name*” for all your tasks to GPU compute nodes. *Partition name: NA100q, PA100q, PA400q, RTXA6Kq, DGxq...*

- n - number of tasks to execute. Optional. By default, 1 task.

- c - cores to allocate per each task. For hyper-threaded tasks. Optional. By default, 1 core per task.

- N - number of servers to allocate. Optional. By default, number of servers will be selected by scheduler.

- gres=gpu:1 or --gres=gpu:2 - number of GPUs to allocate for the job. **Must be used if your job is using GPU**

sinfo - view information about **Slurm** nodes and partitions

Example:

```
$ sinfo
```

squeue – show current tasks in queue

Example:

```
$ squeue
```

sacct – show previous jobs

Example:

```
$ sacct
```

scancel - Kill a job

Example:

\$ scancel xx (*xx=job id*)

scontrol – show job details

Example:

\$ scontrol show job xx (*xx=job id*)

3. Loading and Unloading Modules

3.1 Cluster environment has certain modules pre-compiled and ready to be used – different versions of Cuda toolkit, MPI, different compilers and so on.

Use the following command to list the default modules added to user environment (.bashrc file in user home folder):

\$ module list

3.2 To see all available modules type

\$ module avail

In the list there are two kinds of modules: • local modules, which are specific to the node, or head node only • shared modules, which are made available from a shared storage, and which only become available for loading after the shared module is loaded.

3.3 Modules can be loaded using the add or load options. A list of modules can be added by spacing them:

\$ module add cuda11.1/toolkit/xxxx

By loading a certain module – system will add certain variables to your environment and modify your PATH/ LD_LIBRARY_PATH variables.

To add module to your environment permanently – modify your .bashrc file accordingly.

4. srun

srun is suitable for a simple task and for tasks that don't take long to complete - for which you would like to receive an output to your SSH terminal.

4.1 Simple job with srun

Simplest srun command be:

```
$ srun -p Partition name -n 1 abc.out (you can use command "sinfo" to list the Partition name)
```

e.g. `$ srun -p NA100q -n 1 abc.out`

For instance:

e.g. `$ srun -p NA100q -n 1 hostname`

srun: Job step created

node01

4.2 srun job with GPU

If your job is using GPU, please **DO use** "--gres=gpu:N" (replace N with actual number of GPUs your task needs) option. Otherwise, your job will fail or even worse, impact performance of other running jobs by sharing the GPU:

To be able to run GPU job, you need to have CUDA libraries in your environment. You need to run module load to load CUDA into your environment.

To verify current list of loaded modules, use "**module list**" command.

```
$ module list
```

Currently Loaded Modulefiles:

1) slurm/slurm/21.08.6

```
$ srun -p PA100q -n 1 --gres=gpu:1
```

```
/cm/shared/apps/cuda11.1/sdk/current/7_CUDALibraries/simpleCUBLAS/simpleCUBLAS
```

srun: job 7357 queued and waiting for resources

srun: job 7357 has been allocated resources

*/cm/shared/apps/cuda11.1/sdk/current/7_CUDALibraries/simpleCUBLAS/simpleCUBLAS:
error while loading shared libraries: libcublas.so.11:*

CUDA module is not there, you need to add it with "**module load cuda11.1/toolkit**", otherwise your job could fail due to missing cuda libraries:

```
$ module available (show available software package)
```

```
$ module load cuda11.1/toolkit/11.1.1
```

```
$ module load cuda11.1/blas/11.1.1
```

```
$ module list
```

Currently Loaded Modulefiles:

1) slurm/slurm/21.08.6 2) **cuda11.1/toolkit/11.1.1** 3) **cuda11.1/blas/11.1.1**

```
$ srun -p PA100q -n 1 --gres=gpu:1
```

```
/cm/shared/apps/cuda11.1/sdk/current/7_CUDA Libraries/simpleCUBLAS/simpleCUBLAS
```

srun: job xxxx queued and waiting for resources

srun: job xxxx has been allocated resources

GPU Device 0: "Ampere" with compute capability 8.0

simpleCUBLAS test running..

simpleCUBLAS test passed.

4.3 srun job with several cores and several GPUs

For each 4 cores that you use (either 4 tasks with single core or single task with 4 cores) please use 1 GPU. Try to stick to this allocation blocks: 4 cores and 1 GPU. For instance if you plan to use 8 cores and not sure how many GPUs you need – 1/2/3/4, use 2:

Please note that “nbody” here is not MPI binary and it is mentioned here only as srun example. For such **non-mpi** task normally it does not make sense to use more than “-n 1” since all these executions will be completely separate, unaware of each other:

```
$ srun -p partition name -n 8 -c 1 --gres=gpu:2
```

```
/cm/shared/apps/cuda11.1/sdk/current/7_CUDA Libraries/simpleCUBLAS/simpleCUBLAS
```

5. sbatch

5.1 Simple job with sbatch

Create the following file (**testjob.sh**):

```
#!/bin/sh
```

```
#SBATCH -p partition name
```

If you want to use dedicate node, add in (#SBATCH -w nodelist)

```
#SBATCH -n 1
```

```
hostname
```

Submit the job with sbatch. Output will be in slurm-<JOBID>.out:

```
$ sbatch testjob.sh
```

```
Submitted batch job xxx
```

```
$ cat slurm-xxx.out
```

```
Hostname (e.g. node01)
```

5.2 sbatch job with GPU (sample testscript locate at /export/home2/test/testgpu.sh)

Create the following file (testgpu.sh):

```
#!/bin/sh
```

```
#SBATCH -o gpu-job-%j.output
```

```
#SBATCH -p NA100q
```

```
#SBATCH --gres=gpu:1
```

```
#SBATCH -n 1
```

```
module load cuda11.1/toolkit
```

```
module load cuda11.1/blas/11.1.1
```

```
/cm/shared/apps/cuda11.1/sdk/current/7_CUDALibraries/simpleCUBLAS/simpleCUBLAS
```

Submit the job with sbatch. Verify the queue with squeue. Output will be in gpu-job-%j.output

```
$ sbatch testgpu.sh
```

```
Submitted batch job xxx
```

```
$ squeue
```

```
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
xxx NA100q testgpujob. username R 0:02 1 node01
```

5.3 sbatch job with several cores and GPUs

Create the following file:

```
#!/bin/sh
```

```
#SBATCH -p gpu
```

```
#SBATCH --gres=gpu:1
```

```
#SBATCH -n 4
```

```
module load cuda11.1/toolkit
```

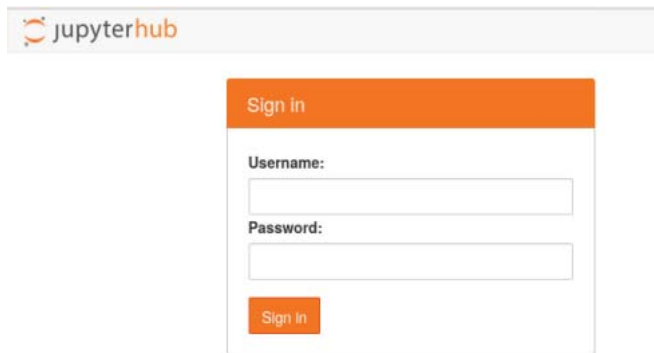
```
srun /path/to/your/program
```

Submit the job with sbatch. This will run will 4 instances of your program, all of them will utilize 1 GPU. The same job can be run interactively (output is directed to your SSH terminal instead of slurm-<JOBID>.out file) with the following srun command line:

```
$ srun -p gpu -n 4 --gres=gpu:1 /path/to/your/program
```

6. Jupyter

The Jupyter login host is at an IP address of 155.69.144.18 then the URL the user uses to log in from with a browser (figure below) is by default: <https://155.69.144.18:8000>



6.1 Jupyter Notebook Examples

The default Jupyter provides a number of machine learning notebook examples that can be executed with Jupyter. The notebooks include some applications developed with TensorFlow, PyTorch, MXNet, and other frameworks. The applications can be found in the /cm/shared/examples/jupyter/notebooks/ directory:

```
$ ls /cm/shared/examples/jupyter/notebooks/
```

```
Keras+TensorFlow2-addition.ipynb  R-iris.ipynb  TensorFlow-minigo.ipynb
MXNet-superresolution.ipynb        Spark-pipeline.ipynb  Pytorch-cartpole.ipynb
Spark+XGBoost-mortgage.ipynb
```

The datasets needed to execute these notebooks can be found in the `/cm/shared/examples/jupyter/datasets/` directory:

```
$ ls /cm/shared/examples/jupyter/datasets/
```

```
880f8b8a6fd-mortgage-small.tar.gz kaggle-iris.csv
```

Users can copy these examples to their home directories, create or choose appropriate kernels to execute them, and interactively run them from Jupyter. In order to edit notebooks, the write permissions must be kept during the copy. It is the responsibility of users to make sure that the required modules are loaded by their Jupyter kernels.

7. MPI Examples

7.1 OpenMPI sbatch job

Sample MPI program (helloworld.c):

```
/* test of MPI */
#include "mpi.h"
#include <stdio.h>
#include <string.h>

int main(int argc, char **argv)
{
    char idstr[2232]; char buff[22128];
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int numprocs; int myid; int i; int namelen;
    MPI_Status stat;

    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&myid);
    MPI_Get_processor_name(processor_name, &namelen);

    if(myid == 0)
    {
        printf("WE have %d processors\n", numprocs);
        for(i=1;i<numprocs;i++)
        {
            sprintf(buff, "Hello %d", i);
```



```

    MPI_Send(buff, 128, MPI_CHAR, i, 0, MPI_COMM_WORLD); }
    for(i=1;i<numprocs;i++)
    {
        MPI_Recv(buff, 128, MPI_CHAR, i, 0, MPI_COMM_WORLD, &stat);
        printf("%s\n", buff);
    }
}
else
{
    MPI_Recv(buff, 128, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &stat);
    sprintf(idstr, " Processor %d at node %s ", myid, processor_name);
    strcat(buff, idstr);
    strcat(buff, "reporting for duty\n");
    MPI_Send(buff, 128, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
}
MPI_Finalize();
}

```

Load OpenMPI module and compile the program:

\$ module list

Currently Loaded Modulefiles:

1) cuda11.1/toolkit/11.1.1 2) slurm/slurm/21.08.6

\$ module load openmpi/gcc/64/4.1.2

\$ mpicc helloworld.c

\$ ls a.out

a.out

Create batch file (helloworld.sh):

#!/bin/sh

#SBATCH -p PA100q

#SBATCH -n 4

#SBATCH -N 2

#SBATCH --ntasks-per-node=2

module load openmpi/gcc/64/4.1.2

mpirun a.out

```
$ sbatch helloworld.sh
Submitted batch job xxx
```

```
$ cat slurm-xxx.out
WE have 4 processors
Hello 1 Processor 1 at node node02 reporting for duty
Hello 2 Processor 2 at node node03 reporting for duty
Hello 3 Processor 3 at node node04 reporting for duty
```

7.2 MVapich2 srun job

For srun and sbatch command, the **ONLY MPI that will work is MVAPICH2**
Make sure to load it:

```
$ module list
Currently Loaded Modulefiles:
  1) cuda11.1/toolkit/11.1.1  2) slurm/slurm/21.08.6  3) openmpi/gcc/64/4.1.2
$ module unload openmpi/gcc/64/4.1.2
$ module load mvapich2/gcc/64/2.3.4
$ module list
Currently Loaded Modulefiles:
  1) cuda11.1/toolkit/11.1.1  2) slurm/slurm/21.08.6  3) openmpi/gcc/64/4.1.2
```

Compile the task and run it:

```
$ mpicc helloworld.c -o helloworld
$ ls helloworld
helloworld
```

```
$ srun -p PA100q -n 4 helloworld
srun: Job step created
WE have 4 processors
Hello 1 Processor 1 at node node02 reporting for duty
Hello 2 Processor 2 at node node03 reporting for duty
Hello 3 Processor 3 at node node04 reporting for duty
```

Here is an example of **non-working MPI task – most probably wrong MPI module was loaded here, either during runtime or during compilation:**

```
$ srun -p PA100q -n 4 helloworld
srun: Job step created
WE have 1 processors
WE have 1 processors
```

WE have 1 processors
WE have 1 processors

Here you can see that each task is launched separately. They are unaware of each other and each thinks it has 1 processor.

7.3 MVapich2 sbatch job

Create following file (helloworld.sh):

```
-----  
#!/bin/sh  
  
#SBATCH -p PA100q  
#SBATCH -n 4  
#SBATCH -N 2  
#SBATCH --ntasks-per-node=2  
  
module load mvapich2/gcc/64/2.3.4  
  
srun helloworld  
-----
```

Submit the job and verify results:

```
$ sbatch helloworld.sh  
Submitted batch job xxx
```

```
$ cat slurm-xxx.out
```

```
WE have 4 processors  
Hello 1 Processor 1 at node node02 reporting for duty  
Hello 2 Processor 2 at node node03 reporting for duty  
Hello 3 Processor 3 at node node04 reporting for duty
```